White Rose Research Online

eprints@whiterose.ac.uk
https://eprints.whiterose.ac.uk
Universities of Leeds, Sheffield and York

Deposited via The University of Sheffield.

White Rose Research Online URL for this paper:
https://eprints.whiterose.ac.uk/id/eprint/128429/

Version: Accepted Version

---

**Article:**
Dehghantanha, A., Azmoodeh , A. and Choo, K.-K.R. (2019) Robust Malware Detection for Internet Of (Battlefield) Things Devices Using Deep Eigenspace Learning. IEEE Transactions on Sustainable Computing, 4 (1). pp. 88-95. ISSN: 2377-3782

https://doi.org/10.1109/TSUSC.2018.2809665

---

**Reuse**
Items deposited in White Rose Research Online are protected by copyright, with all rights reserved unless indicated otherwise. They may be downloaded and/or printed for private study, or other acts as permitted by national copyright laws. The publisher or other rights holders may allow further reproduction and re-use of the full text version. This is indicated by the licence information on the White Rose Research Online record for the item.

**Takedown**
If you consider content in White Rose Research Online to be in breach of UK law, please notify us by emailing eprints@whiterose.ac.uk including the URL of the record and the reason for the withdrawal request.

UNIVERSITY OF LEEDS
University of Sheffield
UNIVERSITY of York

# Robust Malware Detection for Internet Of (Battlefield) Things Devices Using Deep Eigenspace Learning

Amin Azmoodeh, Ali Dehghantanha, *Senior Member, IEEE,*
and Kim-Kwang Raymond Choo, *Senior Member, IEEE,*

**Abstract**—Internet of Things (IoT) in military setting generally consists of a diverse range of Internet-connected devices and nodes (e.g. medical devices to wearable combat uniforms), which are a valuable target for cyber criminals, particularly state-sponsored or nation state actors. A common attack vector is the use of malware. In this paper, we present a deep learning based method to detect Internet Of Battlefield Things (IoBT) malware via the device's Operational Code (OpCode) sequence. We transmute OpCodes into a vector space and apply a deep Eigenspace learning approach to classify malicious and bening application. We also demonstrate the robustness of our proposed approach in malware detection and its sustainability against junk code insertion attacks. Lastly, we make available our malware sample on Github, which hopefully will benefit future research efforts (e.g. for evaluation of proposed malware detection approaches).

**Index Terms**—Internet of Things Malware, Internet Of Battlefield Things, Malware Detection, Deep Eigenspace Learning, Deep Learning, Machine Learning

## 1 Introduction

A typical Internet of Things (IoT) deployment includes a wide pervasive network of (smart) Internet-connected devices, Internet-connected vehicles, embedded systems, sensors, etc. that autonomously sense, store, transfer and process collected data [1], [2], [3]. IoT devices in a civilian setting includes health [4], agriculture [5], smart city [6], and energy and transport management systems [7], [8]. Furthermore, the IoT and its capabilities causes an increasing interest in leveraging the IoT's advantages and features to improve combat ability in battlefields and managing war resources. Utilizing IoT technology in military operations and defensive applications is referred to Internet Of Battlefield Things(IoBT) [9], [10].

There are underpinning security and privacy concerns in such IoT environment [1], [11], [12], [13]. While IoT and IoBT share many of the underpinning cyber security risks (e.g. malware infection [14]), the sensitive nature of IoBT deployment (e.g. military and warfare) makes IoBT architecture and devices more likely to be targeted by cyber criminals. In addition, actors who target IoBT devices and infrastructure are more likely to be state-sponsored, better resourced, and professionally trained.

- *Amin Azmoodeh is with the Department of Electrical and Computer Engineering, Shiraz University, Shiraz, Iran.*
- *Ali Dehghantanha is with the Department of Computer Science, School of Computing, Science & Engineering, University of Salford, Greater Manchester, UK*
- *Kim-Kwang Raymond Choo is with the Department of Information Systems and Cyber Security, The University of Texas at San Antonio, San Antonio, TX 78249, USA (email: raymond.choo@fulbrightmail.org)*

*Manuscript received July 28, 2017; revised X, 2017.*

Intrusion and malware detection and prevention are active research areas [15], [16], [17], [18], [19], [20], but due to the resource constrained hardware, nature of IoT and IoBT devices and customized operating systems existing solutions are unlikely to be suited for real-world deployment. Majority of IoT malware misuse low-level vulnerabilities of compromised device to infect. Thus, it is necessary to answer the need for IoT and IoBT specific malware detection [20].

There has been recent interest in utilizing machine learning and deep learning techniques in malware detection (e.g. distinguishing between malware and benign applications), due to their potential for increased accuracy and robustness [15], [21], [22], [23]. Typically, the following criteria are used to evaluate the utility of machine learning and deep learning techniques in malware detection:

- True Positive (TP): indicates that a malware is correctly identified as a malicious application.
- True Negative (TN): indicates that a benign is detected as a non-malicious application correctly.
- False Positive (FP): indicates that a benign is falsely detected as a malicious application.
- False Negative (FN): indicates that a malware is not detected and labeled as a non-malicious application.

Based on the above criteria, the following metrics will then be used to quantify a given system:

**Accuracy** is the number of samples that a classifier correctly detects, divided by the number of all malware and goodware applications:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

**Precision** is the ratio of predicted malware that are correctly labeled a malware, and is defined as follows:

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

**Recall** or detection rate is the ratio of malware samples that are correctly predicted, and is defined as follows:

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

**F-Measure** is the harmonic mean of precision and recall, and is defined as follows:

$$F - Measure = \frac{2 * TP}{2 * TP + FP + FN} \quad (4)$$

Cross-validation is a fundamental technique in machine learning to assess the extent that the findings of an experiment can be generalized into an independent dataset. While there are many cross validation techniques (e.g. Leave-P-Out, K-fold and Repeated Random Sub-sampling), when the size of a dataset is limited K-fold validation techniques (e.g. 10-fold) are generally used. K-Fold validation techniques are also commonly used to validate the fitness of a model to a hypothetical validation set in the absence of an independent validation set [24], [25].

Due to the fast pace of malware development and the significant increase in the number of malware samples, using deep learning techniques for malware detection is gaining prominence. For example, Yuan et al. [26] used Deep Belief Network on a combination of static and dynamic features of Android APKs and reportedly achieved 96.76% of overall accuracy and 95.77% and 97.84% of precision and recall respectively in malware detection. Saxe and Berlin [27] presented a detection method using deep feed-forward neural network using a set of Windows program features including Byte/Entropy histogram, portable executable (PE) data and metadata and printable character sequence and reportedly achieved detection and false positive rates of 95% and 0.1% respectively.

Bilar [28] introduced Operational Codes (OpCodes) as a suitable and reliable feature for malware identification using machine learning techniques. Moskovitch et al. [29] applied different text mining techniques to detect candidate features of Windows malware sample OpCode for classification using algorithms such as Neural Network and Decision Tree and reportedly attained 94.43% accuracy. Santos et al. [30] leveraged the frequency of a specific OpCode appearance in benign and malicious Windows applications as a feature for Decision Tree, Support Vector Machine, Bayesian Networks and K-Nearest Neighbors algorithms and achieved 95.90% accuracy in malware detection. Hashemi et al [31] extracted OpCodes of Windows benign and malware executable files and then form a graph for each sample and turned the generated graph into a vector using Power Iteration procedure to train classifiers such as Support Vector Machine and Adaboost. They achieved an accuracy and a F-measure of 96.09% and 95.98% respectively. Siddiqui et al. [32] utilized occurrence frequency and principle component analysis on a dataset of Windows malware and goodware application OpCodes and obtained 93.1% detection rate using Random Forest as the classifier.

Thus, we posit that OpCode analysis may provide a solid basis for a robust and sustainable deep learning based IoT and IoBT malware detection system. This is a challenge that has not been addressed in the literature. Specifically, in this paper, we extract OpCode sequence of 1078 benignware and 128 malware (all ARM compatible IoT applications). We utilize Class-Wise Information Gain technique for class aware feature selection [33]. Then, the selected features (OpCodes) of each sample are converted into a graph in which OpCodes are represented by the graph's nodes while the graph edges represent the nodes' affinity in disassembled file of each sample. Finally, generated graphs of malicious and benign samples are used for classification of IoT and IoBT malware and goodware applications using Eigenspace and deep convolutional networks techniques. We achieve 99.68% accuracy in detecting malware samples, with precision and recall rates of 98.59% and 98.37% respectively.

To the best of our knowledge, this is the first OpCode-based deep learning method for IoT and IoBT malware detection. We then demonstrate the robustness of our proposed approach, against existing OpCode based malware detection systems in [30], [31]. We also demonstrate the sustainability of our proposed approach against junk-code insertion attacks. Specifically, our proposed approach employs a class-wise feature selection technique to overrule less important OpCodes in order to resist junk-code insertion attacks. Furthermore, we leverage all the elements of Eigenspace to increase detection rate and sustainability. Finally, as a secondary contribution, we share a normalized dataset of IoT malware and benign applications[1], which may be used by fellow researchers to evaluate and benchmark future malware detection approaches. On the other hand, since the proposed method belongs to OpCode based detection category, it could be adaptable for non-IoT platforms.

In the next section, we briefly review related work. Section 3 describes our collection, preprocessing and evaluation methodology. Section 4 presents our proposed approach, followed by its evaluation in Section 5. Section 6 concludes this paper and suggests several future research agenda.

## 2 Related Literature

Malware detection methods can be broadly categorized into static and dynamic analysis [34]. In dynamic malware detection approaches, the program is executed in a controlled environment (e.g. a virtual machine or a sandbox) to collect its behavioral attributes such as required resources, execution path, and requested privilege, in order to classify a program as malware or benign [35], [36], [37]. Static approaches (e.g. signature-based detection, byte-sequence n-gram analysis, opcode sequence identification and control flow graph traversal) statically inspect a program code to detect suspicious applications.

David et al [38] proposed a framework, $Deepsign$, to automatically detect malware using a signature generation method. The latter creates a dataset based on behaviour logs of API calls, registry entries, web searches, port accesses, etc,

1. The samples are available on https://github.com/azmoodeh/IoTMalwareDetection, where the benign samples are in binary and the malware samples are in OpCode.

in a sandbox and then converts logs to a binary vector. They used deep belief network for classification and reportedly achieved 98.6% accuracy. In another study, Pascanu et al. [39] proposed a method to model malware execution using natural language modeling. They extracted relevant features using recurrent neural network to predict the next API calls. Then, both logistic regression and multi-layer perceptrons were applied as the classification module on next API call prediction and using history of past events as features. It was reported that 98.3% true positive rate and 0.1% false positive rate were achieved.

Demme et al. [40] examined the feasibility of building a malware detector in IoT nodes' hardware using performance counters as a learning feature and K-Nearest Neighbor, Decision Tree and Random Forest as classifiers. The reported accuracy rate for different malware family ranges from 25% to 100%. Alam et al. [41] applied Random Forest on a dataset of Internet-connected smartphone devices to recognize malicious codes. They executed APKs in an Android emulator and recorded different features such as memory information, permission and network for classification and evaluated their approach using different tree sizes. Their findings showed that the optimal classifier contains 40 trees, and 0.0171 of mean square root was attained.

In order to detect crypto-ransomware on Android devices as management nodes of an IoT networks, Azmoodeh et al. [42] recorded the power usage of running processes and identified distinguishable local energy consumption patterns for benign and ransomware. They broke down the power usage pattern into sub-samples and classified them, as well as aggregating sub-samples' labels to determine final label. The proposed approach reportedly achieved 92.75% accuracy. Securing IoT backbone against malware attacks motivated Haddad Pajouh et al. [43] to propose a two-layer dimension reduction and two-tier classification module to detect malicious activities. Specifically, the authors used Principle Component Analysis and Linear Discrimination Analysis to reduce the dataset and then used Naïve Bayes and K-Nearest Neighbor to classify samples. They achieved detection and false alarm rates of 84.86% and 4.86%, respectively.

While OpCodes are considered an efficient feature for malware detection, there does not appear to have been any attempt to use OpCodes for IoT and IoBT malware detection. In addition, using deep learning for robust malware detection in IoT networks appears to be another understudied topic. Thus, in this paper, we seek to contribute to this gap by exploring the potential of using OpCodes as features for malware detection with deep Eigenspace learning.

## 3 Dataset Creation and Feature Selection

We created a dataset of 1078 benign and 128 malware samples for ARM-based IoT applications [44]. All malware samples were collected using VirusTotal[2] Threat Intelligence platform between February 2015 and January 2017. All goodware were collected from a variety of official IoT App stores such as Pi Store [3].

2. http://www.virustotal.com
3. https://thepihut.com/collections/raspberry-pi-store

IoT and IoBT application are likely to consist of a long sequence of OpCodes, which are instructions to be performed on device processing unit. In order to disassemble samples, we utilized Objdump (GNU binutils version 2.27.90) as a disassembler to extract the OpCodes. Creating n-gram OpCode sequence is a common approach to classify malware based on their disassembled codes [45], [46]. The number of rudimentary features for length $N$ is $C^N$, where $C$ is the size of instruction set. It is clear that a large increase in $N$ will result in feature explosion. In addition, decreasing the size of feature increases robustness and effectiveness of detection because ineffective features reduce performance of machine learning approach. Therefore, there is a tendency to first apply a feature selection algorithm and find the best features [47] in order to reduce the feature set to avoid feature explosion. Information retrieval techniques are widely used for feature selection [48]. Information Gain (IG) is an information-theoretic approach to select global features by ranking them based on the amount of information content available in a classification problem. IG applies statistical tools to choose global features and does not consider class information. In some situations such as imbalanced datasets, global feature selection methods neglect minor class-specified features which may reduce system efficiency.

Class-Wise Information Gain (CIG) [33] is proposed to overcome global feature selection imperfection and aims to recognize more useful features based on available class information. To understand how $CIG$ is calculated for an arbitrary two-class problem, we refer the reader to Equation 5, where $P(v_f = 1, C_i)$ denotes the probability of feature $f$ appearing in $C_i$, and $P(v_f = 0, C_j)$ is the probability of feature $f$ being absent from $C_j$. $C_B$ and $C_M$ denote benign program and malicious applications, respectively.

$$\begin{aligned} CIG(f, C_B) &= P(v_f = 1, C_B) * log \frac{P(v_f = 1, C_B)}{P(v_f = 1)P(C_B)} \\ &+ P(v_f = 0, C_M) * log \frac{P(v_f = 0, C_M)}{P(v_f = 0)P(C_M)} \\ CIG(f, C_M) &= P(v_f = 1, C_M) * log \frac{P(v_f = 1, C_M)}{P(v_f = 1)P(C_M)} \\ &+ P(v_f = 0, C_B) * log \frac{P(v_f = 0, C_B)}{P(v_f = 0)P(C_B)} \end{aligned} \tag{5}$$

In this study, 4,543 1-gram and 610,109 2-gram distinct OpCode sequences were extracted and $CIG(f, C_B)$ and $CIG(f, C_M)$ were calculated. The top 82 features (approximately 0.01% of all features) $\{f_1, ..., f_{82}\}$ were selected, where $f_i$ belongs to either the 1-gram or 2-gram category. Size of selected feature set is limited to $j = 82$ because there was a significant gap between $f_{j<=82}$'s and $f_{j>=83}$'s CIG values. Due to the high computational resource consumptions, all k-gram($k >= 2$) features were ignored. Features were selected from the 1-gram and 2-gram sequences based on their $CIG(f, C_B)$ and $CIG(f, C_M)$ values. As shown in Figure 1, majority of the malware features are 2-gram sequences and 1-gram sequences constitute a large proportion of benign application features. Such knowledge would be crucial in the development of our IoT and IoBT malware detection method, as discussed in the next section.

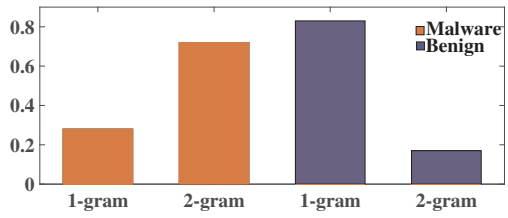



Fig. 1. 1-Gram and 2-Gram Feature Distribution For Benign and Malware Samples

## 4 Proposed Approach

Our proposed method is illustrated in Fig.2, and consists of two phases, namely: OpCode-Sequence Graph Generation phase and Deep Eigensapce Learning phase. Also, feature selection phase is included in Fig.2.

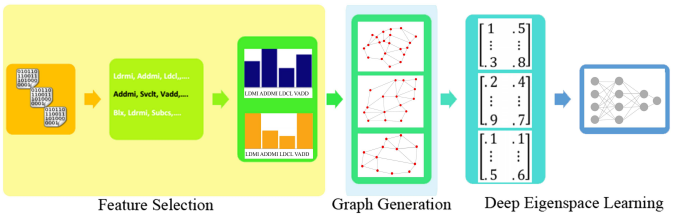



Fig. 2. Proposed Approach

### 4.1 Opcode-Sequence Graph Generation

Control Flow Graph (CFG) is a data structure that represents the order of OpCodes in an executable file. A graph, $G = \langle V, E \rangle$, has two sets: $V$ and $E$. $V$ denotes the graph's vertices and $E_{i,j}$ shows the relation between $V_i$ and $V_j$. Previous research has shown the usefulness of this representation in malware detection [30], [31], [49]. $V_i \epsilon \{f_j | j = 1, ..., 82\}$ are vertices and the edges' values represent the relation between vertices (features).

In order to construct the OpCodes' graph, edge values should be computed. The general approach for calculating $E_{i,j}$ value is to increment $E_{i,j}$ by 1 when $V_i$ occurs immediately after $V_j$ in the sample's OpCode sequence. Utilizing this procedure would lead to generation of an adjacency matrix for each sample application within our dataset. Furthermore, normalization of matrix rows would turn $E_{i,j}$ values into probability of occurrence of $V_i$. Then, all $V_j$ and $E_{i,j}$s values are normalized to a value between 0 and 1. Considering the situations in which $V_i$ and $V_j$ are placed exactly together neglect the longer distance of OpCodes' neighborhood. In other words, merely observing a specific order of OpCodes leads to a crisp representation of OpCode sequence in a graph.

However, the Crisp approach for computing $E_{i,j}$ has its own drawbacks. Applying feature selection and then incrementing $E_{i,j}$ by 1 for exact OpCode's occupants result in a sparse adjacency matrix, which may poorly represent a sample file that is not suitable for a classification task. In addition, malware developers may inject some useless junk OpCode(s), such as $NOP$[4] or $(PUSH, POP)$[5] to circumvent/deceive OpCode's neighborhood calculation method.

Therefore, we propose a heuristic criteria shown in Formulation(6) to calculate the graph edge values. Fundamental elements of Formulation(6) is the distance between OpCodes. A longer distance increases the divisor exponentially and consequently produces a smaller $E_{i,j}$. To improve Formulation(6) by spotting distance mitigates the drawbacks of calculating edges by immediate occurrence and highlights the effect of OpCodes distance. $\alpha$ is a tuning parameter to adjust the impact of OpCode's distance. In this study, we let $\alpha = 1$ to have $E_{i,j} = 1$ for exactly adjacent OpCodes similar to Hashemi et al. [31] approach. Also, $\alpha$ can control the effect of OpCodes' distance in detection rate. Formulation(6) would produce a graph of 82 vertices for each given malware and benign sample as the learning material for Deep Eigensapce Learning phase of our method. Figure 3 illustrates the output of Opcode-Sequence Graph Generation phase for a sample. For instance, the edge's value between $OpCode_i = call$ and $OpCode_j = sub$ means that sum of all $E_{i,j}$s calculated by Formulation(6) is 0.2.

$$E_{i,j} = \sum_{s \epsilon S} \frac{2}{1 + \alpha * e^{min(|s-t-1|)}}$$
$$S = \{index\ of\ all\ appearance\ of\ OpCode_{V_i} \\ in\ sample's\ OpCode\ sequence\}$$
$$t \epsilon \{index\ of\ all\ appearance\ of\ OpCode_{V_j} \\ in\ sample's\ OpCode\ sequence\} \quad (6)$$

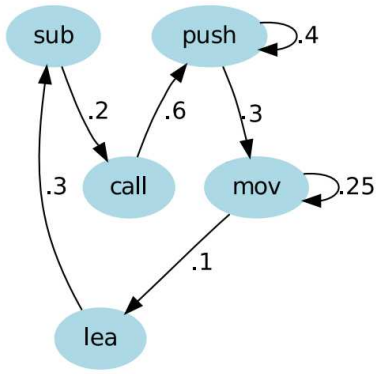



Fig. 3. A schematic view of the sample's generated graph

### 4.2 Deep Eigenspace Learning

#### 4.2.1 Eigenspace

Graphs as a complex data structure for representing relations between vertices are a prevalent data type in machine learning. There are very few data mining and deep learning algorithms [50] that accept a graph as an input [51]. Therefore, a possible alternative is to embed a graph into a vector

4. No Operation
5. After execution of these two OpCodes, the state of program is similar to before executing the PUSH instruction

space [52]. Indeed, graph embedding is a bridge between statistical pattern recognition and graph mining.

Eigenvectors and eigenvalue are two characteristic elements in the graph's spectrum [53], which could linearly transform a graph's adjacency matrix into a vector space (see Equation 7). $v$ ,$\lambda$ and $A$ denote eigenvectors, eigenvalues and a graph's adjacency or an affinity matrix respectively. In this paper, we employ a subset of $v$ and $\lambda$ for the learning phase.

$$Av = \lambda v \tag{7}$$

To obtain a tangible knowledge of the generated CGFs' structure, a graph that illustrates the cumulative of all samples in our dataset is created (see Figure 4). Figure 4 consists of two major diagonal building blocks (marked with red borders), which indicates that two main data distributions exist in the given samples. Based on the graph's spectrum theory, in this condition, there should be an explicit eigengap in the matrix's eigenvalues [54], and Figure 5 depicts the existence of a gap between $\lambda_2$ and $\lambda_k (k > 2)$. Hence,

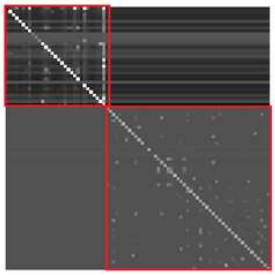


Fig. 4. An Overview of Samples' Cumulation Affinity Matrix

two first eigenvectors of sample's matrix($v_1$ and $v_2$ ) include much more information about the matrix compared to the remaining eigenvectors, and could represent the whole matrix appropriately.

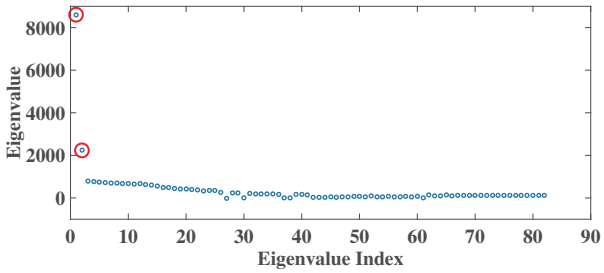



Fig. 5. Sample's Cumulation (Figure 4) Eigenvalues

Moreover, in the learning phase, due to different data distribution of eigenvalues for malware and benign samples, $\lambda_1$ and $\lambda_2$ are utilized alongside $v_1$ and $v_2$ to detect a sample label and increase our method performance. Figures 6 and 7 illustrate the difference between malware and benign eigenvalues' ($\lambda_{1,2}$) data distribution and indicate suitability of employing $\lambda_1$ and $\lambda_2$ as features for a classification task.

#### 4.2.2 Deep Learning

Deep Learning (DL) [55], [56] or deep structured learning is an evolved version of Neural Networks (NN) [57]. A

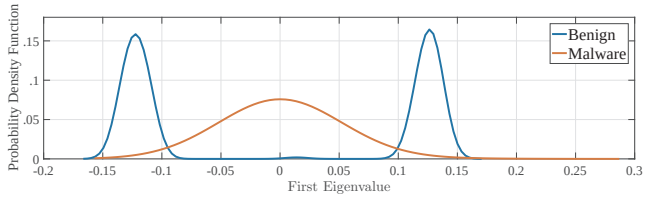



Fig. 6. First Eigenvalue ($\lambda_1$) Distribution

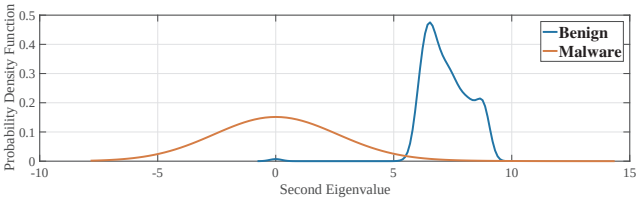



Fig. 7. Second Eigenvalue ($\lambda_2$) Distribution

standard NN includes few or many simple, inter-connected nodes called neurons. NN's neurons are organized in a few layers, namely: an input layer, several hidden layers and an output layer. DL as an "upgraded" trend of NN, focuses on deeper data structure learning by concentrating on the hidden layer's abilities and functionalities. Deep learning has recently been successfully deployed to address challenges in a variety of applications, such as speech recognition and machine vision [58], [59]. There are different variations of DL such as Convolutional Networks, Restricted Boltzmann Machines, and Sparse Coding [60].

In this paper, a Convolutional Network is used as the deep learning module of our proposed approach because of its potential for accurate classification in the presence of complex and non-linear data patterns [61]. The first two eigenvectors ($v_1$ and $v_2$) and eigenvalues ($\lambda_1$ and $\lambda_2$) of the samples are used as input values for classification.

## 5 Findings

In this section, we evaluate the accuracy, precision, recall and F-measure of our proposed approach, in order to demonstrate its robustness in detecting IoT and IoBT malware. Moreover, we demonstrate the sustainability of our proposed approach against junk code insertion attacks.

### 5.1 Robustness

To show the robustness of our proposed approach and benchmark it against existing proposals, two congruent algorithms [30], [31] described in Section 1 are applied on our generated dataset using Adaboost [62] as the classification algorithm. All evaluations were conducted using MATLAB R2015a running on a Microsoft Windows 10 Pro personal computer powered by Intel Core i7 2.67GHz and 8GB RAM.

A 10-fold cross validation was used in the validating, and the comparative summary is presented in Table 1. It is clear that our proposed approach outperforms the proposals of Hashemi et al. [31] and Santos et al. [30]. Santos et al. [30] is a basic and commonly-known OpCode based malware detection algorithm and Hashemi et al. [31] is the most

analogous OpCode based approach regarding the utilizing eigenspace.

Accuracy is a general criteria for evaluating performance of an algorithm for both malware and benign class identification. The proposed approach achieves a high accuracy of 99.68%, while the approaches of Hashemi et al. [31] and Santos et al. [30] achieve 98.59% and 95.91% accuracy respectively. Recall or detection rate is an important criteria and the proposed approach achieves 98.37%, in comparison to 81.55% and 77.70% for the other two approaches.

Our proposed approach also outperforms the approaches of Hashemi et al. [31] and Santos et al. [30] in terms of precision rate and F-Measure. Utilizing class-wise feature selection aids beneficial features of minor class to be more effective during classification phase. Also, using Formulation(6) to calculate OpCode's distance leads to represent more OpCode sequence patterns in sample's graph. Moreover, employing deep neural networks for classification leads to superior classifier.

| | Accuracy | Precision | Recall | F-Measure |
|---|---|---|---|---|
| Proposed Method | 99.68% | 98.59% | 98.37% | 98.48% |
| Hashemi et al. [31] | 96.87% | 91.09% | 81.55% | 86.05% |
| Santos et al. [30] | 95.91% | 86.25% | 77.70% | 81.75% |

TABLE 1
Performance Evaluation Of Detection Methods on Our Dataset

### 5.2 Sustainability Against Junk Code Insertion Attacks

Junk code injection attack is a malware anti-forensic technique against OpCode inspection. Junk code insertion may include addition of benign OpCode sequences, which never run in a malware or inclusion of instructions (e.g. NOP) that do not actually make any difference in malware activities. Junk code insertion technique would obfuscate malicious OpCode sequences and reduce the balance of malicious OpCodes in a malware [63].

In our proposed approach, we use an affinity based criteria to evade junk OpCode injection anti-forensics technique. Our feature selection method eliminates less instructive OpCodes to mitigate the effects of injecting junk OpCodes.

To demonstrate sustainability of our proposed approach against code insertion attack, in an iterative manner, a specified proportion({5%, 10%, 15%, 20%, 25%, 30%}) of all elements in each sample's generated graph were selected randomly and their value incremented by one. For example, in the 4th iteration of the evaluations, 20% of the indices in each sample's graph were chosen to increment their value by one. In addition, in our evaluations the possibility of a repetitive element selection was included to simulate injecting an OpCode more than once. Incrementing $E_{i,j}$ in the sample's generated graph is equivalent to injecting $OpCode_j$ next to the $OpCode_i$ in a sample's instruction sequence to mislead detection algorithm. Algorithm 1 describes an iteration of junk code insertion during experiments and it is necessary to mention this procedure should repeat for each iteration of k-fold validation.

**Algorithm 1** Junk Code Insertion Procedure

**Input:** Trained Classifier $D$, Test Samples $S$, Junk Code Percentage $k$
**Output:** Predicted Class for Test Samples $P$

$P = \{\}$
**for** each $sample$ in $S$ **do**
  $W$= Compute the CFG of $sample$ based on Section 4.1
  $R$ = {select $k\%$ of $W$'s index randomly(Allow duplicate indices)}
  **for** each $index$ in $R$ **do**
    $W_{index} = W_{index} + 1$
  **end for**
  Normalize $W$
  $e_1, e_2$= 1st and 2nd eigenvectors of $W$
  $l_1, l_2$= 1st and 2nd eigenvalues of $W$
  $P = P \bigcup D(e_1, e_2, l_1, l_2)$
**end for**
**return** $P$

Figure 8 reports on the performance of the proposed approach over the stated condition. It is clear that our proposed approach achieves an acceptable performance in the face of an junk code insertion attack. A steady trend for Accuracy indicates that truly classified samples significantly outnumber false positive and negative samples – see Equation 1. Furthermore, comparing Precision and Recall results imply that false positive rate or false alarm rate is greater than the false negative rate based on Equations 2 and 3 and taking into account Recall's stable graph, the proposed approach is robust against junk code insertion.

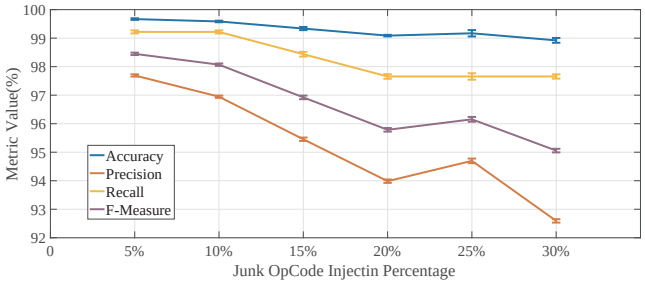



Fig. 8. Evaluation of Proposed Method Robustness Over Junk Code Insertion

## 6 Concluding Remarks

IoT, particularly IoBT, will be increasingly important in the foreseeable future. While no malware detection solution is foolproof and there will always be a constant race between cyber attackers and cyber defenders, it is important that we maintain persistent pressure on threat actors [64].

In this paper, we presented an IoT and IoBT malware detection approach based on class-wise selection of OpCodes sequence as a feature for classification task. A graph of selected features was created for each sample and a deep Eigenspace learning approach was used for malware classification. Our evaluations demonstrated the robustness of our approach in malware detection with an accuracy of 98.37% and a precision rate of 98.59%, as well as the capability to mitigate junk code insertion attacks.

In the future, we plan to evaluate our approach against larger and broader datasets and implementing a prototype of the proposed approach in a real-world IoT and IoBT system for evaluation and refinement.

## ACKNOWLEDGMENTS

We thank VirusTotal for providing us a private API key to access their data for constructing our dataset. This work is partially supported by the European Council International Incoming Fellowship (FP7-PEOPLE-2013-IIF) grant.

## REFERENCES

[1] E. Bertino, K.-K. R. Choo, D. Georgakopolous, and S. Nepal, "Internet of things (iot): Smart and secure service delivery," *ACM Transactions on Internet Technology*, vol. 16, no. 4, p. Article No. 22, 2016.

[2] X. Li, J. Niu, S. Kumari, F. Wu, A. K. Sangaiah, and K.-K. R. Choo, "A three-factor anonymous authentication scheme for wireless sensor networks in internet of things environments," *Journal of Network and Computer Applications*, 2017.

[3] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of things (iot): A vision, architectural elements, and future directions," *Future generation computer systems*, vol. 29, no. 7, pp. 1645–1660, 2013.

[4] F. Leu, C. Ko, I. You, K.-K. R. Choo, and C.-L. Ho, "A smartphone-based wearable sensors for monitoring real-time physiological data," *Computers & Electrical Engineering*, 2017.

[5] M. Roopaei, P. Rad, and K.-K. R. Choo, "Cloud of things in smart agriculture: Intelligent irrigation monitoring by thermal imaging," *IEEE Cloud Computing*, vol. 4, no. 1, pp. 10–15, 2017.

[6] X. Li, J. Niu, S. Kumari, F. Wu, and K.-K. R. Choo, "A robust biometrics based three-factor authentication scheme for global mobility networks in smart city," *Future Generation Computer Systems*, 2017.

[7] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Computer networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[8] D. Miorandi, S. Sicari, F. De Pellegrini, and I. Chlamtac, "Internet of things: Vision, applications and research challenges," *Ad Hoc Networks*, vol. 10, no. 7, pp. 1497–1516, 2012.

[9] A. Kott, A. Swami, and B. J. West, "The internet of battle things," *Computer*, vol. 49, no. 12, pp. 70–75, 2016.

[10] M. J. Farooq and Q. Zhu, "Secure and reconfigurable network design for critical information dissemination in the internet of battlefield things (iobt)," *arXiv preprint arXiv:1703.01224*, 2017.

[11] C. Tankard, "The security issues of the internet of things," *Computer Fraud & Security*, vol. 2015, no. 9, pp. 11 – 14, 2015.

[12] C. J. DOrazio, K. K. R. Choo, and L. T. Yang, "Data exfiltration from internet of things devices: ios devices as case studies," *IEEE Internet of Things Journal*, vol. 4, no. 2, pp. 524–535, April 2017.

[13] S. Watson and A. Dehghantanha, "Digital forensics: the missing piece of the internet of things promise," *Computer Fraud & Security*, vol. 2016, no. 6, pp. 5–8, 2016.

[14] E. Bertino and N. Islam, "Botnets and internet of things security," *Computer*, vol. 50, no. 2, pp. 76–79, Feb 2017.

[15] J. Gardiner and S. Nagaraja, "On the security of machine learning in malware c&c detection: A survey," *ACM Computing Surveys*, vol. 49, no. 3, p. Article No. 59, 2016.

[16] J. Peng, K.-K. R. Choo, and H. Ashman, "User profiling in intrusion detection: A review," *Journal of Network and Computer Applications*, vol. 72, pp. 14–27, 2016.

[17] E. M. Rudd, A. Rozsa, M. Gnther, and T. E. Boult, "A survey of stealth malware attacks, mitigation measures, and steps toward autonomous open world solutions," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 2, pp. 1145–1172, 2016.

[18] S. Iqbal, M. L. M. Kiah, B. Dhaghighi, M. Hussain, S. Khan, M. K. Khan, and K.-K. R. Choo, "On cloud security attacks: A taxonomy and intrusion detection and prevention as a service," *Journal of Network and Computer Applications*, vol. 77, pp. 98–120, 2016.

[19] Y. Ye, T. Li, D. Adjeroh, and S. S. Iyengar, "A survey on malware detection using data mining techniques," *ACM Computing Surveys*, vol. 50, no. 3, p. Article No. 41, 2017.

[20] Z. K. Zhang, M. C. Y. Cho, C. W. Wang, C. W. Hsu, C. K. Chen, and S. Shieh, "Iot security: Ongoing challenges and research opportunities," in *2014 IEEE 7th International Conference on Service-Oriented Computing and Applications*, Nov 2014, pp. 230–234.

[21] P. Faruki, A. Bharmal, V. Laxmi, V. Ganmoor, M. S. Gaur, M. Conti, and M. Rajarajan, "Android security: A survey of issues, malware penetration, and defenses," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 2, pp. 998–1022, Secondquarter 2015.

[22] Z. Fadlullah, F. Tang, B. Mao, N. Kato, O. Akashi, T. Inoue, and K. Mizutani, "State-of-the-art deep learning: Evolving machine intelligence toward tomorrows intelligent network traffic control systems," *IEEE Communications Surveys & Tutorials*, 2017.

[23] N. Milosevic, A. Dehghantanha, and K.-K. R. Choo, "Machine learning aided android malware classification," *Computers & Electrical Engineering*, 2017.

[24] R. Kohavi *et al.*, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Ijcai*, vol. 14(2). Stanford, CA, 1995, pp. 1137–1145.

[25] Y. Bengio and Y. Grandvalet, "No unbiased estimator of the variance of k-fold cross-validation," *Journal of machine learning research*, vol. 5, no. Sep, pp. 1089–1105, 2004.

[26] Z. Yuan, Y. Lu, and Y. Xue, "Droiddetector: android malware characterization and detection using deep learning," *Tsinghua Science and Technology*, vol. 21, no. 1, pp. 114–123, Feb 2016.

[27] J. Saxe and K. Berlin, "Deep neural network based malware detection using two dimensional binary program features," in *Malicious and Unwanted Software (MALWARE), 2015 10th International Conference on*, 2015, pp. 11–20.

[28] D. Bilar, "Opcodes as predictor for malware," *Int. J. Electron. Secur. Digit. Forensic*, vol. 1, no. 2, pp. 156–168, Jan. 2007.

[29] R. Moskovitch, C. Feher, N. Tzachar, E. Berger, M. Gitelman, S. Dolev, and Y. Elovici, "Unknown malcode detection using opcode representation," *Intelligence and Security Informatics*, pp. 204–215, 2008.

[30] I. Santos, F. Brezo, X. Ugarte-Pedrero, and P. G. Bringas, "Opcode sequences as representation of executables for data-mining-based unknown malware detection," *Information Sciences*, vol. 231, pp. 64–82, 2013.

[31] H. Hashemi, A. Azmoodeh, A. Hamzeh, and S. Hashemi, "Graph embedding as a new approach for unknown malware detection," *Journal of Computer Virology and Hacking Techniques*, 2016.

[32] M. Siddiqui, M. C. Wang, and J. Lee, "Data mining methods for malware detection using instruction sequences," in *Proceedings of the 26th IASTED International Conference on Artificial Intelligence and Applications*, ser. AIA '08. Anaheim, CA, USA: ACTA Press, 2008, pp. 358–363.

[33] Y. Tan, *Class-Wise Information Gain*. John Wiley & Sons, Inc., 2016, ch. 11, pp. 150–172.

[34] K. Shaerpour, A. Dehghantanha, and R. Mahmod, "Trends in android malware detection," *The Journal of Digital Forensics, Security and Law: JDFSL*, vol. 8, no. 3, p. 21, 2013.

[35] N. Idika and A. P. Mathur, "A survey of malware detection techniques," *Purdue University*, vol. 48, 2007.

[36] P. Vinod, R. Jaipur, V. Laxmi, and M. Gaur, "Survey on malware detection methods," in *Proceedings of the 3rd Hackers Workshop on computer and internet security (IITKHACK09)*, 2009, pp. 74–79.

[37] Z. Bazrafshan, H. Hashemi, S. M. H. Fard, and A. Hamzeh, "A survey on heuristic malware detection techniques," in *Information and Knowledge Technology (IKT), 2013 5th Conference on*. IEEE, 2013, pp. 113–120.

[38] O. E. David and N. S. Netanyahu, "Deepsign: Deep learning for automatic malware signature generation and classification," in *Neural Networks (IJCNN), 2015 International Joint Conference on*. IEEE, 2015, pp. 1–8.

[39] R. Pascanu, J. W. Stokes, H. Sanossian, M. Marinescu, and A. Thomas, "Malware classification with recurrent networks," in *Acoustics, Speech and Signal Processing (ICASSP), 2015 IEEE International Conference on*. IEEE, 2015, pp. 1916–1920.

[40] J. Demme, M. Maycock, J. Schmitz, A. Tang, A. Waksman, S. Sethumadhavan, and S. Stolfo, "On the feasibility of online malware detection with performance counters," in *ACM SIGARCH Computer Architecture News*, vol. 41, no. 3. ACM, 2013, pp. 559–570.

[41] M. S. Alam and S. T. Vuong, "Random forest classification for detecting android malware," in *2013 IEEE International Conference on Green Computing and Communications and IEEE Internet of Things and IEEE Cyber, Physical and Social Computing*, Aug 2013, pp. 663–669.

[42] A. Azmoodeh, A. Dehghantanha, M. Conti, and K.-K. R. Choo, "Detecting crypto-ransomware in iot networks based on energy consumption footprint," *Journal of Ambient Intelligence and Humanized Computing*, 2017.

[43] H. H. Pajouh, R. Javidan, R. Khayami, D. Ali, and K. K. R. Choo, "A two-layer dimension reduction and two-tier classification model for anomaly-based intrusion detection in iot backbone networks," *IEEE Transactions on Emerging Topics in Computing*, vol. PP, no. 99, pp. 1–1, 2016.

[44] D. Brash, "Recent additions to the armv7-a architecture," in *2010 IEEE International Conference on Computer Design*, Oct 2010, pp. XIX–XIX.

[45] D. K. S. Reddy and A. K. Pujari, "N-gram analysis for computer virus detection," *Journal in Computer Virology*, vol. 2, no. 3, pp. 231–239, 2006.

[46] A. Shabtai, R. Moskovitch, C. Feher, S. Dolev, and Y. Elovici, "Detecting unknown malicious code by applying classification techniques on opcode patterns," *Security Informatics*, vol. 1, no. 1, p. 1, 2012.

[47] A. Feizollah, N. B. Anuar, R. Salleh, and A. W. A. Wahab, "A review on feature selection in mobile malware detection," *Digit. Investig.*, vol. 13, no. C, pp. 22–37, Jun. 2015.

[48] G. Forman, "An extensive empirical study of feature selection metrics for text classification," *Journal of machine learning research*, vol. 3, no. Mar, pp. 1289–1305, 2003.

[49] B. Anderson, D. Quist, J. Neil, C. Storlie, and T. Lane, "Graph-based malware detection using dynamic analysis," *Journal in Computer Virology*, vol. 7, no. 4, pp. 247–258, 2011.

[50] D. J. Cook and L. B. Holder, *Mining graph data*. John Wiley & Sons, 2006.

[51] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern classification*. John Wiley & Sons, 2012.

[52] K. Riesen and H. Bunke, *Graph classification and clustering based on vector space embedding*. World Scientific, 2010, vol. 77.

[53] F. R. Chung, *Spectral graph theory*. American Mathematical Soc., 1997, no. 92.

[54] M. Newman, "mathematics of networks," in *The New Palgrave Dictionary of Economics*, S. N. Durlauf and L. E. Blume, Eds. Basingstoke: Palgrave Macmillan, 2008.

[55] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural networks*, vol. 61, pp. 85–117, 2015.

[56] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[57] S. S. Haykin, S. S. Haykin, S. S. Haykin, and S. S. Haykin, *Neural networks and learning machines*. Pearson Upper Saddle River, NJ, USA:, 2009, vol. 3.

[58] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, http://www.deeplearningbook.org.

[59] L. Deng, D. Yu *et al.*, "Deep learning: methods and applications," *Foundations and Trends® in Signal Processing*, vol. 7, no. 3–4, pp. 197–387, 2014.

[60] P. Druzhkov and V. Kustikova, "A survey of deep learning methods and software tools for image classification and object detection," *Pattern Recognition and Image Analysis*, vol. 26, no. 1, p. 9, 2016.

[61] S. Mallat, "Understanding deep convolutional networks," *Phil. Trans. R. Soc. A*, vol. 374, no. 2065, p. 20150203, 2016.

[62] Y. Freund and R. E. Schapire, "A desicion-theoretic generalization of on-line learning and an application to boosting," in *European conference on computational learning theory*. Springer, 1995, pp. 23–37.

[63] A. Walenstein and A. Lakhotia, "The software similarity problem in malware analysis," in *Dagstuhl Seminar Proceedings*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2007.

[64] K.-K. R. Choo, *A Conceptual Interdisciplinary Plug-and-Play Cyber Security Framework*. Boston, MA: Springer US, 2014, pp. 81–99.




**Amin Azmoodeh** received his B.S.degree in Computer Engineering and M.Sc. degree in Artificial Intelligence from Shiraz University. His main research interests are theory of Machine Learning and Artificial Intelligence and its applications especially in cybersecurity and digital forensics. He has several years' experience in analyzing and implementing security mechanism in Enterprise Resource Planning software.



**Ali Dehghantanha** (SM17) is a Marie-Curie International Incoming Fellow in Cyber Forensics and a fellow of the UK Higher Education Academy (HEA). He has served for many years in a variety of research and industrial positions. Other than Ph.D. in Cyber Security he holds many professional certificates such as GXPN, GREM, CISM, CISSP, and CCFP. He has served as an expert witness, cyber forensics analysts and malware researcher with leading players in Cyber-Security and E-Commerce.



**Kim-Kwang Raymond Choo** (SM15) received the Ph.D. in Information Security from Queensland University of Technology, Australia. He currently holds the Cloud Technology Endowed Professorship at The University of Texas at San Antonio, and is a Fellow of the Australian Computer Society. He serves on the editorial board of Cluster Computing, Digital Investigation, IEEE Access, IEEE Cloud Computing, Future Generation Computer Systems, Journal of Network and Computer Applications, PLoS ONE, etc. He also serves as the Special Issue Guest Editor of ACM Transactions on Embedded Computing Systems (2017; DOI: 10.1145/3015662), ACM Transactions on Internet Technology (2016; DOI: 10.1145/3013520), Digital Investigation (2016; DOI: 10.1016/j.diin.2016.08.003), Future Generation Computer Systems (2016; DOI: 10.1016/j.future.2016.04.017), IEEE Cloud Computing (2015; DOI: 10.1109/MCC.2015.84), IEEE Network (2016; DOI: 10.1109/MNET.2016.7764272), IEEE Transactions on Dependable and Secure Computing (2017; DOI: 10.1109/TDSC.2017.2664183), Journal of Computer and System Sciences (2017; DOI: 10.1016/j.jcss.2016.09.001), Multimedia Tools and Applications (2017; DOI: 10.1007/s11042-016-4081-z), Personal and Ubiquitous Computing (2017; DOI: 10.1007/s00779-017-1043-z), Pervasive and Mobile Computing (2016; DOI: 10.1016/j.pmcj.2016.10.003), Wireless Personal Communications (2017; DOI: 10.1007/s11277-017-4278-0) etc. He was named Cybersecurity Educator of the Year APAC (2016 Cybersecurity Excellence Awards are produced in cooperation with the Information Security Community on LinkedIn) in 2016. In 2015, he and his team won the Digital Forensics Research Challenge organized by Germanys University of Erlangen-Nuremberg. He was named one of 10 Emerging Leaders in the Innovation category of The Weekend Australian Magazine/Microsofts Next 100 series in 2009, and his other awards include ESORICS 2015 Best Research Paper Award, Highly Commended Award from Australia New Zealand Policing Advisory Agency (2014), Fulbright Scholarship (2009), 2008 Australia Day Achievement Medallion, and British Computer Societys Wilkes Award (2007).